

Step 0 — Imports & Setup

```
In [19]: # =====  
# ARTI 404 - Image Processing  
# Lab 3 - Student Procedural Manual  
# =====  
  
import cv2  
import numpy as np  
import matplotlib.pyplot as plt  
import os  
  
# Make sure the images folder exists  
os.makedirs('images', exist_ok=True)  
  
print("OpenCV version:", cv2.__version__)  
print("NumPy version:", np.__version__)
```

OpenCV version: 5.0.0

NumPy version: 2.3.4

Step#1 — Read an image using OpenCV

```
In [20]: # Step#1: Read an image using the OpenCV library  
import cv2  
  
img = cv2.imread('./images/input.jpg')  
  
if img is None:  
    print("❌ Error: Image not found. Check the path.")  
else:  
    print("✅ Image loaded successfully")  
    print("Shape:", img.shape)  
    print("Data type:", type(img))
```

✅ Image loaded successfully

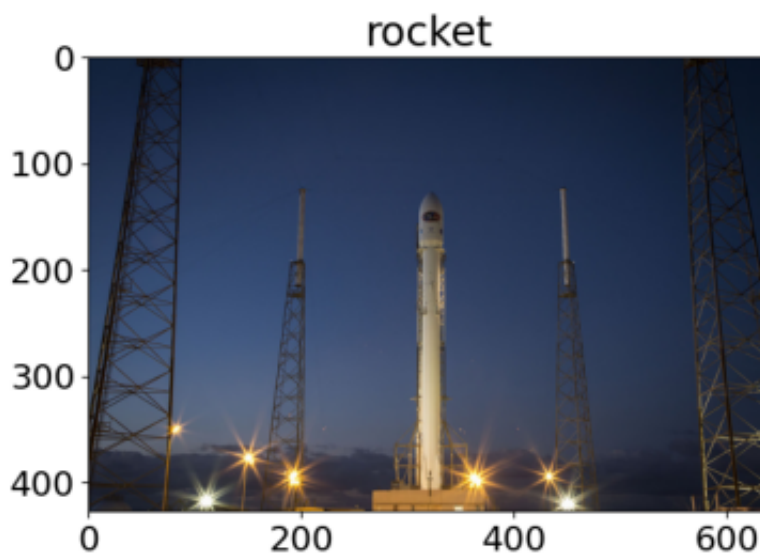
Shape: (480, 640, 3)

Data type: <class 'numpy.ndarray'>

```
In [21]: # Display the image inside Jupyter
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

plt.figure(figsize=(6, 4))
plt.imshow(img_rgb)
plt.title('Input Image')
plt.axis('off')
plt.show()
```

Input Image



```
In [22]: import cv2
img = cv2.imread('./images/input.jpg')
print(type(img))
```

```
<class 'numpy.ndarray'>
```

Step#2 — Load an image in Grayscale

```
In [6]: # Step#2: Load an image in grayscale mode
import cv2

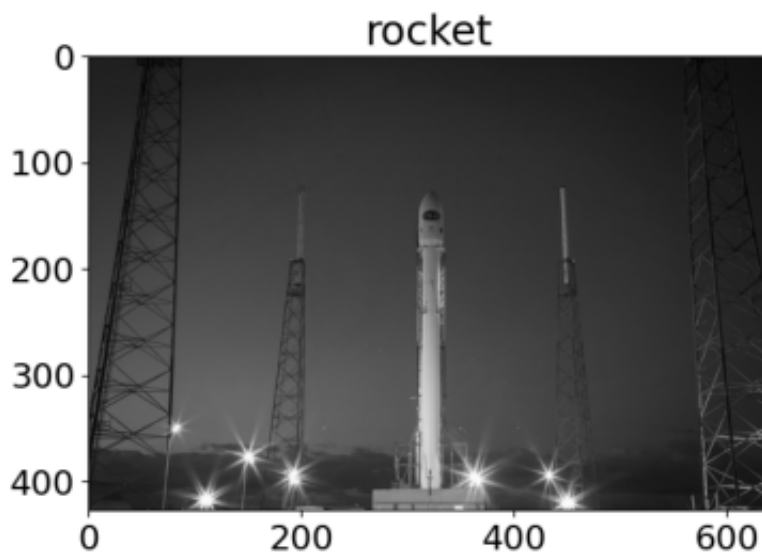
gray_img = cv2.imread('images/input.jpg', cv2.IMREAD_GRAYSCALE)

print("Gray image shape:", gray_img.shape)

plt.figure(figsize=(6, 4))
plt.imshow(gray_img, cmap='gray')
plt.title('Grayscale Image (IMREAD_GRAYSCALE)')
plt.axis('off')
plt.show()
```

Gray image shape: (480, 640)

Grayscale Image (IMREAD_GRAYSCALE)



Step#3 — Save an image

```
In [7]: # Step#3: Save an image
import cv2

gray_img = cv2.imread('images/input.jpg', cv2.IMREAD_GRAYSCALE)

success = cv2.imwrite('images/output.jpg', gray_img)

if success:
    print("✅ Image saved to images/output.jpg")
else:
    print("❌ Failed to save the image.")
```

✅ Image saved to images/output.jpg

Step4 — Convert an image to different color spaces

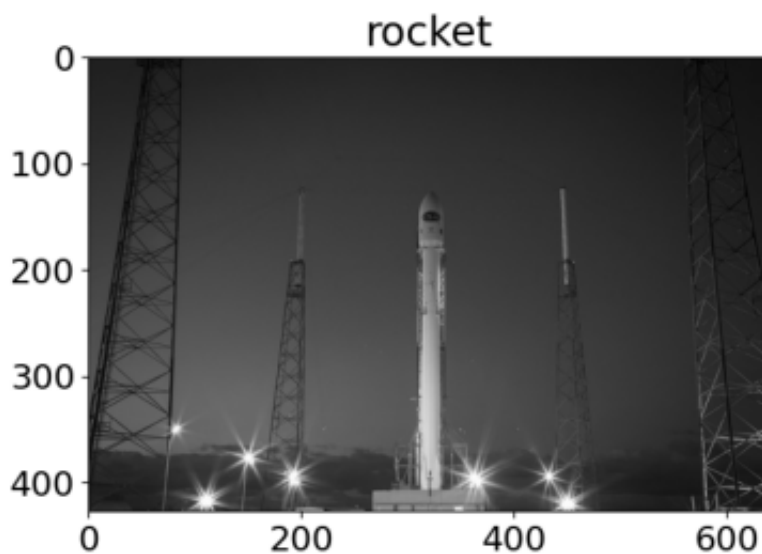
```
In [8]: # Step#4 (a): Convert to Grayscale using cvtColor
import cv2

img = cv2.imread('images/input.jpg')
gray_img = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

plt.figure(figsize=(6, 4))
plt.imshow(gray_img, cmap='gray')
plt.title('Grayscale (cvtColor)')
plt.axis('off')
plt.show()

cv2.imwrite('images/gray_cvtColor.jpg', gray_img)
print("✅ Saved: images/gray_cvtColor.jpg")
```

Grayscale (cvtColor)



✓ Saved: images/gray_cvtColor.jpg

```
In [9]: # Step#4 (b): List all available COLOR_ flags
import cv2

color_flags = [x for x in dir(cv2) if x.startswith('COLOR_')]
print("Number of color conversions available:", len(color_flags))
print(color_flags)
```

```
Number of color conversions available: 374
['COLOR_BAYER_BG2BGR', 'COLOR_BAYER_BG2BGRA', 'COLOR_BAYER_BG2BGR_EA',
'COLOR_BAYER_BG2BGR_VNG', 'COLOR_BAYER_BG2GRAY', 'COLOR_BAYER_BG2RGB',
'COLOR_BAYER_BG2RGBA', 'COLOR_BAYER_BG2RGB_EA',
'COLOR_BAYER_BG2RGB_VNG', 'COLOR_BAYER_BGGR2BGR',
'COLOR_BAYER_BGGR2BGRA', 'COLOR_BAYER_BGGR2BGR_EA',
'COLOR_BAYER_BGGR2BGR_VNG', 'COLOR_BAYER_BGGR2GRAY',
'COLOR_BAYER_BGGR2RGB', 'COLOR_BAYER_BGGR2RGBA',
'COLOR_BAYER_BGGR2RGB_EA', 'COLOR_BAYER_BGGR2RGB_VNG',
'COLOR_BAYER_GB2BGR', 'COLOR_BAYER_GB2BGRA', 'COLOR_BAYER_GB2BGR_EA',
'COLOR_BAYER_GB2BGR_VNG', 'COLOR_BAYER_GB2GRAY', 'COLOR_BAYER_GB2RGB',
'COLOR_BAYER_GB2RGBA', 'COLOR_BAYER_GB2RGB_EA',
'COLOR_BAYER_GB2RGB_VNG', 'COLOR_BAYER_GBRG2BGR',
'COLOR_BAYER_GBRG2BGRA', 'COLOR_BAYER_GBRG2BGR_EA',
'COLOR_BAYER_GBRG2BGR_VNG', 'COLOR_BAYER_GBRG2GRAY',
'COLOR_BAYER_GBRG2RGB', 'COLOR_BAYER_GBRG2RGBA',
'COLOR_BAYER_GBRG2RGB_EA', 'COLOR_BAYER_GBRG2RGB_VNG',
'COLOR_BAYER_GR2BGR', 'COLOR_BAYER_GR2BGRA', 'COLOR_BAYER_GR2BGR_EA',
'COLOR_BAYER_GR2BGR_VNG', 'COLOR_BAYER_GR2GRAY', 'COLOR_BAYER_GR2RGB',
'COLOR_BAYER_GR2RGBA', 'COLOR_BAYER_GR2RGB_EA',
'COLOR_BAYER_GR2RGB_VNG', 'COLOR_BAYER_GRGB2BGR',
```

'COLOR_BAYER_GRBG2BGR', 'COLOR_BAYER_GRBG2BGR_EA',
'COLOR_BAYER_GRBG2BGR_VNG', 'COLOR_BAYER_GRBG2GRAY',
'COLOR_BAYER_GRBG2RGB', 'COLOR_BAYER_GRBG2RGBA',
'COLOR_BAYER_GRBG2RGB_EA', 'COLOR_BAYER_GRBG2RGB_VNG',
'COLOR_BAYER_RG2BGR', 'COLOR_BAYER_RG2BGR_EA', 'COLOR_BAYER_RG2BGR_VNG',
'COLOR_BAYER_RG2GRAY', 'COLOR_BAYER_RG2RGB',
'COLOR_BAYER_RG2RGBA', 'COLOR_BAYER_RG2RGB_EA',
'COLOR_BAYER_RG2RGB_VNG', 'COLOR_BAYER_RGGB2BGR',
'COLOR_BAYER_RGGB2BGR_EA', 'COLOR_BAYER_RGGB2BGR_VNG',
'COLOR_BAYER_RGGB2GRAY',
'COLOR_BAYER_RGGB2RGB', 'COLOR_BAYER_RGGB2RGBA',
'COLOR_BAYER_RGGB2RGB_EA', 'COLOR_BAYER_RGGB2RGB_VNG',
'COLOR_BGR2BGR555', 'COLOR_BGR2BGR565', 'COLOR_BGR2BGR',
'COLOR_BGR2GRAY', 'COLOR_BGR2HLS', 'COLOR_BGR2HLS_FULL',
'COLOR_BGR2HSV', 'COLOR_BGR2HSV_FULL', 'COLOR_BGR2LAB',
'COLOR_BGR2LUV', 'COLOR_BGR2Lab', 'COLOR_BGR2Luv', 'COLOR_BGR2RGB',
'COLOR_BGR2RGBA', 'COLOR_BGR2XYZ', 'COLOR_BGR2YCR_CB',
'COLOR_BGR2YCrCb', 'COLOR_BGR2YUV', 'COLOR_BGR2YUV_I420',
'COLOR_BGR2YUV_IYUV', 'COLOR_BGR2YUV_UYNV', 'COLOR_BGR2YUV_UYVY',
'COLOR_BGR2YUV_Y422', 'COLOR_BGR2YUV_YUNV', 'COLOR_BGR2YUV_YUY2',
'COLOR_BGR2YUV_YUYV', 'COLOR_BGR2YUV_YV12', 'COLOR_BGR2YUV_YVYU',
'COLOR_BGR555BGR', 'COLOR_BGR555BGR_EA', 'COLOR_BGR555GRAY',
'COLOR_BGR555RGB', 'COLOR_BGR555RGBA', 'COLOR_BGR565BGR',
'COLOR_BGR565BGR_EA', 'COLOR_BGR565GRAY', 'COLOR_BGR565RGB',
'COLOR_BGR565RGBA', 'COLOR_BGRA2BGR', 'COLOR_BGRA2BGR555',
'COLOR_BGRA2BGR565', 'COLOR_BGRA2GRAY', 'COLOR_BGRA2RGB',
'COLOR_BGRA2RGBA', 'COLOR_BGRA2YUV_I420', 'COLOR_BGRA2YUV_IYUV',
'COLOR_BGRA2YUV_UYNV', 'COLOR_BGRA2YUV_UYVY', 'COLOR_BGRA2YUV_Y422',
'COLOR_BGRA2YUV_YUNV', 'COLOR_BGRA2YUV_YUY2', 'COLOR_BGRA2YUV_YUYV',
'COLOR_BGRA2YUV_YV12', 'COLOR_BGRA2YUV_YVYU', 'COLOR_BayerBG2BGR',
'COLOR_BayerBG2BGR_EA', 'COLOR_BayerBG2BGR_VNG',
'COLOR_BayerBG2GRAY', 'COLOR_BayerBG2RGB', 'COLOR_BayerBG2RGBA',
'COLOR_BayerBG2RGB_EA', 'COLOR_BayerBG2RGB_VNG',
'COLOR_BayerBGR2BGR', 'COLOR_BayerBGR2BGR_EA',
'COLOR_BayerBGR2BGR_VNG', 'COLOR_BayerBGR2GRAY',
'COLOR_BayerBGR2RGB', 'COLOR_BayerBGR2RGBA',
'COLOR_BayerBGR2RGB_EA', 'COLOR_BayerBGR2RGB_VNG',
'COLOR_BayerGB2BGR', 'COLOR_BayerGB2BGR_EA',
'COLOR_BayerGB2BGR_VNG', 'COLOR_BayerGB2GRAY',
'COLOR_BayerGB2RGB', 'COLOR_BayerGB2RGBA',
'COLOR_BayerGB2RGB_EA', 'COLOR_BayerGB2RGB_VNG',
'COLOR_BayerGBRG2BGR', 'COLOR_BayerGBRG2BGR_EA',
'COLOR_BayerGBRG2BGR_VNG', 'COLOR_BayerGBRG2GRAY',
'COLOR_BayerGBRG2RGB', 'COLOR_BayerGBRG2RGBA',
'COLOR_BayerGBRG2RGB_EA', 'COLOR_BayerGBRG2RGB_VNG',
'COLOR_BayerGR2BGR', 'COLOR_BayerGR2BGR_EA',
'COLOR_BayerGR2BGR_VNG', 'COLOR_BayerGR2GRAY',
'COLOR_BayerGR2RGB', 'COLOR_BayerGR2RGBA',
'COLOR_BayerGR2RGB_EA', 'COLOR_BayerGR2RGB_VNG',
'COLOR_BayerGRBG2BGR', 'COLOR_BayerGRBG2BGR_EA',
'COLOR_BayerGRBG2BGR_VNG', 'COLOR_BayerGRBG2GRAY',
'COLOR_BayerGRBG2RGB', 'COLOR_BayerGRBG2RGBA',
'COLOR_BayerGRBG2RGB_EA', 'COLOR_BayerGRBG2RGB_VNG',

'COLOR_BayerRG2BGR', 'COLOR_BayerRG2BGRA', 'COLOR_BayerRG2BGR_EA',
'COLOR_BayerRG2BGR_VNG', 'COLOR_BayerRG2GRAY', 'COLOR_BayerRG2RGB',
'COLOR_BayerRG2RGBA', 'COLOR_BayerRG2RGB_EA', 'COLOR_BayerRG2RGB_VNG',
'COLOR_BayerRGG2BGR', 'COLOR_BayerRGG2BGRA',
'COLOR_BayerRGG2BGR_EA', 'COLOR_BayerRGG2BGR_VNG',
'COLOR_BayerRGG2GRAY', 'COLOR_BayerRGG2RGB', 'COLOR_BayerRGG2RGBA',
'COLOR_BayerRGG2RGB_EA', 'COLOR_BayerRGG2RGB_VNG',
'COLOR_COLORCVT_MAX', 'COLOR_GRAY2BGR', 'COLOR_GRAY2BGR555',
'COLOR_GRAY2BGR565', 'COLOR_GRAY2BGRA', 'COLOR_GRAY2RGB',
'COLOR_GRAY2RGBA', 'COLOR_HLS2BGR', 'COLOR_HLS2BGR_FULL',
'COLOR_HLS2RGB', 'COLOR_HLS2RGB_FULL', 'COLOR_HSV2BGR',
'COLOR_HSV2BGR_FULL', 'COLOR_HSV2RGB', 'COLOR_HSV2RGB_FULL',
'COLOR_LAB2BGR', 'COLOR_LAB2LBGR', 'COLOR_LAB2LRGB', 'COLOR_LAB2RGB',
'COLOR_LBGR2LAB', 'COLOR_LBGR2LUV', 'COLOR_LBGR2Lab',
'COLOR_LBGR2Luv', 'COLOR_LRGB2LAB', 'COLOR_LRGB2LUV',
'COLOR_LRGB2Lab', 'COLOR_LRGB2Luv', 'COLOR_LUV2BGR', 'COLOR_LUV2LBGR',
'COLOR_LUV2LRGB', 'COLOR_LUV2RGB', 'COLOR_Lab2BGR', 'COLOR_Lab2LBGR',
'COLOR_Lab2LRGB', 'COLOR_Lab2RGB', 'COLOR_Luv2BGR', 'COLOR_Luv2LBGR',
'COLOR_Luv2LRGB', 'COLOR_Luv2RGB', 'COLOR_M_RGBA2RGBA',
'COLOR_RGB2BGR', 'COLOR_RGB2BGR555', 'COLOR_RGB2BGR565',
'COLOR_RGB2BGRA', 'COLOR_RGB2GRAY', 'COLOR_RGB2HLS',
'COLOR_RGB2HLS_FULL', 'COLOR_RGB2HSV', 'COLOR_RGB2HSV_FULL',
'COLOR_RGB2LAB', 'COLOR_RGB2LUV', 'COLOR_RGB2Lab', 'COLOR_RGB2Luv',
'COLOR_RGB2RGBA', 'COLOR_RGB2XYZ', 'COLOR_RGB2YCR_CB',
'COLOR_RGB2YCrCb', 'COLOR_RGB2YUV', 'COLOR_RGB2YUV_I420',
'COLOR_RGB2YUV_IYUV', 'COLOR_RGB2YUV_UYNV', 'COLOR_RGB2YUV_UYVY',
'COLOR_RGB2YUV_Y422', 'COLOR_RGB2YUV_YUNV', 'COLOR_RGB2YUV_YUY2',
'COLOR_RGB2YUV_YUYV', 'COLOR_RGB2YUV_YV12', 'COLOR_RGB2YUV_YVYU',
'COLOR_RGBA2BGR', 'COLOR_RGBA2BGR555', 'COLOR_RGBA2BGR565',
'COLOR_RGBA2BGRA', 'COLOR_RGBA2GRAY', 'COLOR_RGBA2M_RGBA',
'COLOR_RGBA2RGB', 'COLOR_RGBA2YUV_I420', 'COLOR_RGBA2YUV_IYUV',
'COLOR_RGBA2YUV_UYNV', 'COLOR_RGBA2YUV_UYVY', 'COLOR_RGBA2YUV_Y422',
'COLOR_RGBA2YUV_YUNV', 'COLOR_RGBA2YUV_YUY2', 'COLOR_RGBA2YUV_YUYV',
'COLOR_RGBA2YUV_YV12', 'COLOR_RGBA2YUV_YVYU', 'COLOR_RGBA2mRGBA',
'COLOR_XYZ2BGR', 'COLOR_XYZ2RGB', 'COLOR_YCR_CB2BGR',
'COLOR_YCR_CB2RGB', 'COLOR_YCrCb2BGR', 'COLOR_YCrCb2RGB',
'COLOR_YUV2BGR', 'COLOR_YUV2BGRA_I420', 'COLOR_YUV2BGRA_IYUV',
'COLOR_YUV2BGRA_NV12', 'COLOR_YUV2BGRA_NV21', 'COLOR_YUV2BGRA_UYNV',
'COLOR_YUV2BGRA_UYVY', 'COLOR_YUV2BGRA_Y422', 'COLOR_YUV2BGRA_YUNV',
'COLOR_YUV2BGRA_YUY2', 'COLOR_YUV2BGRA_YUYV', 'COLOR_YUV2BGRA_YV12',
'COLOR_YUV2BGRA_YVYU', 'COLOR_YUV2BGR_I420', 'COLOR_YUV2BGR_IYUV',
'COLOR_YUV2BGR_NV12', 'COLOR_YUV2BGR_NV21', 'COLOR_YUV2BGR_UYNV',
'COLOR_YUV2BGR_UYVY', 'COLOR_YUV2BGR_Y422', 'COLOR_YUV2BGR_YUNV',
'COLOR_YUV2BGR_YUY2', 'COLOR_YUV2BGR_YUYV', 'COLOR_YUV2BGR_YV12',
'COLOR_YUV2BGR_YVYU', 'COLOR_YUV2GRAY_420', 'COLOR_YUV2GRAY_I420',
'COLOR_YUV2GRAY_IYUV', 'COLOR_YUV2GRAY_NV12', 'COLOR_YUV2GRAY_NV21',
'COLOR_YUV2GRAY_UYNV', 'COLOR_YUV2GRAY_UYVY', 'COLOR_YUV2GRAY_Y422',
'COLOR_YUV2GRAY_YUNV', 'COLOR_YUV2GRAY_YUY2', 'COLOR_YUV2GRAY_YUYV',
'COLOR_YUV2GRAY_YV12', 'COLOR_YUV2GRAY_YVYU', 'COLOR_YUV2RGB',
'COLOR_YUV2RGBA_I420', 'COLOR_YUV2RGBA_IYUV', 'COLOR_YUV2RGBA_NV12',
'COLOR_YUV2RGBA_NV21', 'COLOR_YUV2RGBA_UYNV', 'COLOR_YUV2RGBA_UYVY',

```
'COLOR_YUV2RGBA_Y422', 'COLOR_YUV2RGBA_YUNV', 'COLOR_YUV2RGBA_YUY2',  
'COLOR_YUV2RGBA_YUYV', 'COLOR_YUV2RGBA_YV12', 'COLOR_YUV2RGBA_YVYU',  
'COLOR_YUV2RGB_I420', 'COLOR_YUV2RGB_IYUV', 'COLOR_YUV2RGB_NV12',  
'COLOR_YUV2RGB_NV21', 'COLOR_YUV2RGB_UYNV', 'COLOR_YUV2RGB_UYVY',  
'COLOR_YUV2RGB_Y422', 'COLOR_YUV2RGB_YUNV', 'COLOR_YUV2RGB_YUY2',  
'COLOR_YUV2RGB_YUYV', 'COLOR_YUV2RGB_YV12', 'COLOR_YUV2RGB_YVYU',  
'COLOR_YUV420P2BGR', 'COLOR_YUV420P2BGRA', 'COLOR_YUV420P2GRAY',  
'COLOR_YUV420P2RGB', 'COLOR_YUV420P2RGBA', 'COLOR_YUV420SP2BGR',  
'COLOR_YUV420SP2BGRA', 'COLOR_YUV420SP2GRAY', 'COLOR_YUV420SP2RGB',  
'COLOR_YUV420SP2RGBA', 'COLOR_YUV420p2BGR', 'COLOR_YUV420p2BGRA',  
'COLOR_YUV420p2GRAY', 'COLOR_YUV420p2RGB', 'COLOR_YUV420p2RGBA',  
'COLOR_YUV420sp2BGR', 'COLOR_YUV420sp2BGRA', 'COLOR_YUV420sp2GRAY',  
'COLOR_YUV420sp2RGB', 'COLOR_YUV420sp2RGBA', 'COLOR_mRGBA2RGBA']
```

```
In [23]: # Step#4 (c): Convert to YUV and separate channels
import cv2

img = cv2.imread('images/input.jpg')

# Convert to YUV
yuv_img = cv2.cvtColor(img, cv2.COLOR_BGR2YUV)

# Display the channels
fig, axes = plt.subplots(1, 3, figsize=(15, 5))

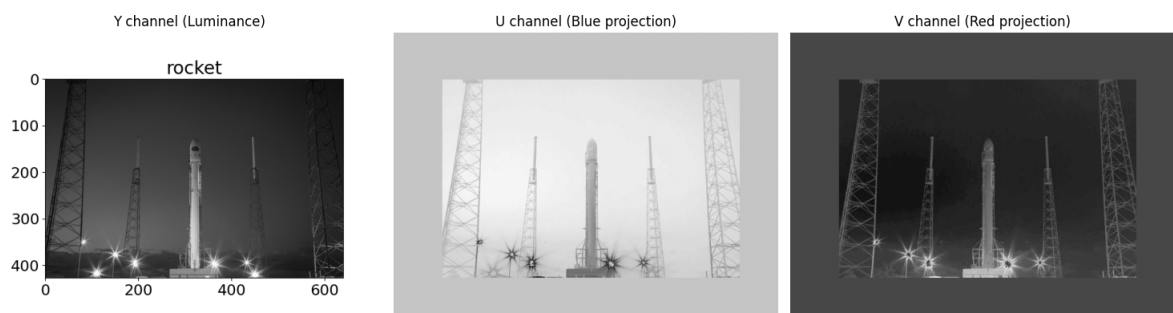
axes[0].imshow(yuv_img[:, :, 0], cmap='gray')
axes[0].set_title('Y channel (Luminance)')
axes[0].axis('off')

axes[1].imshow(yuv_img[:, :, 1], cmap='gray')
axes[1].set_title('U channel (Blue projection)')
axes[1].axis('off')

axes[2].imshow(yuv_img[:, :, 2], cmap='gray')
axes[2].set_title('V channel (Red projection)')
axes[2].axis('off')

plt.tight_layout()
plt.show()

# Save the channels
cv2.imwrite('images/Y_channel.jpg', yuv_img[:, :, 0])
cv2.imwrite('images/U_channel.jpg', yuv_img[:, :, 1])
cv2.imwrite('images/V_channel.jpg', yuv_img[:, :, 2])
print("✅ YUV channels saved")
```



✅ YUV channels saved

Task (Geometric Transformations) — 1) Resize

```
In [24]: # Task - 1) Increase the size of the image
import cv2

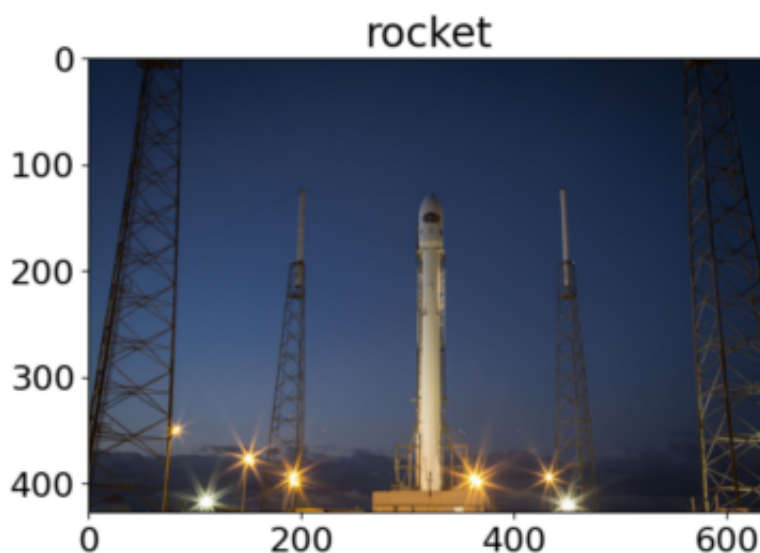
img = cv2.imread('images/input.jpg')
h, w = img.shape[:2]

scale = 1.5    # 150% larger
resized = cv2.resize(img, None, fx=scale, fy=scale,
                      interpolation=cv2.INTER_LINEAR)

# Display
resized_rgb = cv2.cvtColor(resized, cv2.COLOR_BGR2RGB)
plt.figure(figsize=(6, 4))
plt.imshow(resized_rgb)
plt.title(f'Resized {scale}x — New size:
{resized.shape[1]}x{resized.shape[0]}')
plt.axis('off')
plt.show()

cv2.imwrite('images/resized.jpg', resized)
print("Original size:", (w, h))
print("New size:", (resized.shape[1], resized.shape[0]))
```

Resized 1.5x — New size: 960x720



Original size: (640, 480)

New size: (960, 720)

Task - 2) Rotate the image by 120 degrees

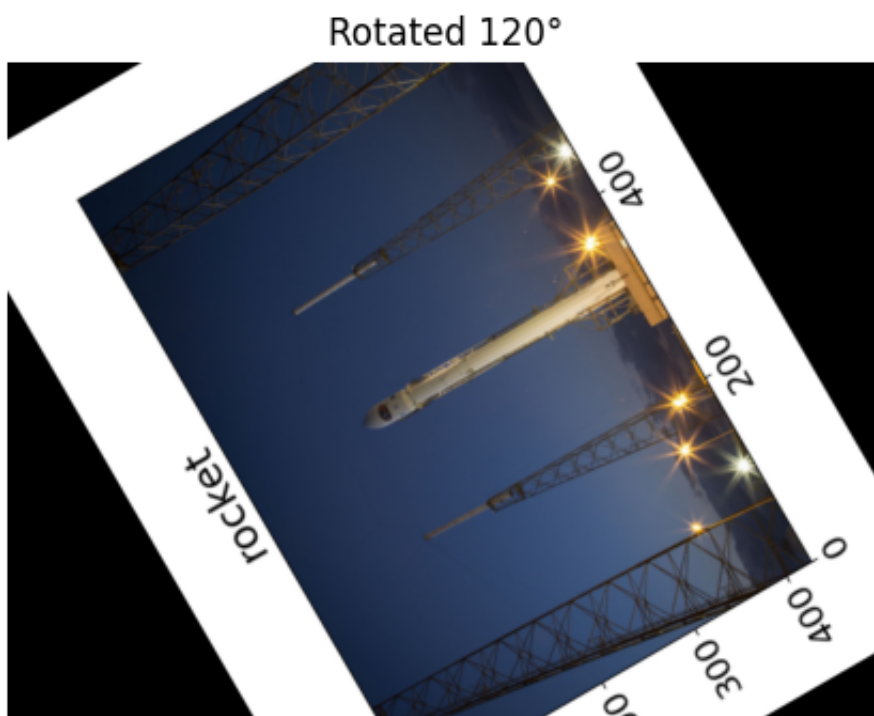
```
In [25]: # Task - 2) Rotate the image by 120 degrees
import cv2

img = cv2.imread('images/input.jpg')
h, w = img.shape[:2]
center = (w // 2, h // 2)

M_rot = cv2.getRotationMatrix2D(center, 120, 1.0) # 120°, scale = 1
rotated = cv2.warpAffine(img, M_rot, (w, h))

rotated_rgb = cv2.cvtColor(rotated, cv2.COLOR_BGR2RGB)
plt.figure(figsize=(6, 4))
plt.imshow(rotated_rgb)
plt.title('Rotated 120°')
plt.axis('off')
plt.show()

cv2.imwrite('images/rotated_120.jpg', rotated)
print("✅ Image rotated by 120 degrees")
```



✅ Image rotated by 120 degrees

Task (Geometric Transformations) — 3) Sheer

```

In [26]: # Task - 3) Perform Shear operations over the image
import cv2
import numpy as np

img = cv2.imread('images/input.jpg')
h, w = img.shape[:2]

# Horizontal shear (x) by 0.5
M_shear_x = np.float32([[1, 0.5, 0],
                        [0, 1, 0]])
shear_x = cv2.warpAffine(img, M_shear_x, (int(w * 1.5), h))

# Vertical shear (y) by 0.5
M_shear_y = np.float32([[1, 0, 0],
                        [0.5, 1, 0]])
shear_y = cv2.warpAffine(img, M_shear_y, (w, int(h * 1.5)))

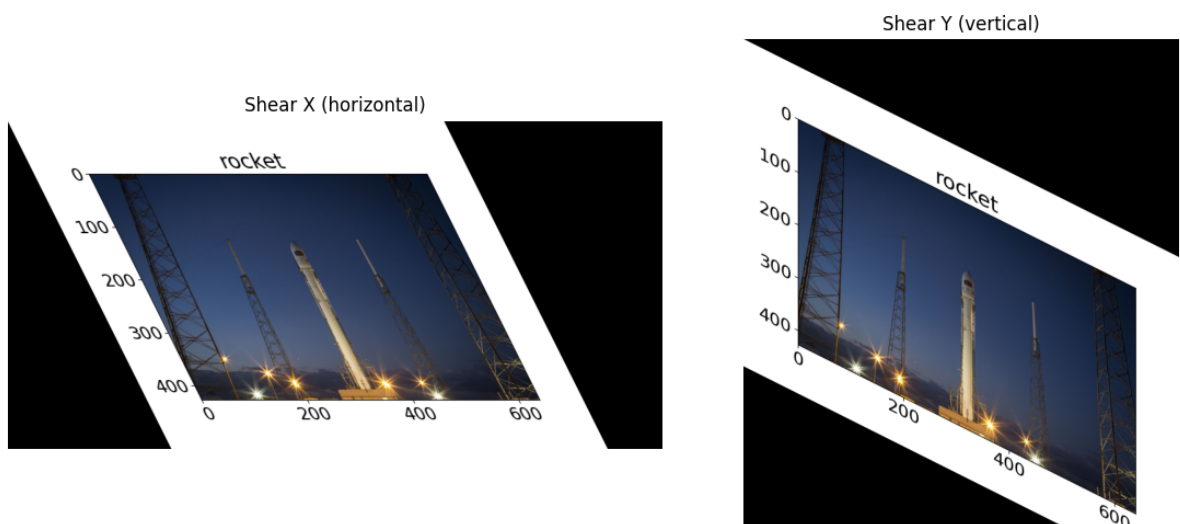
# Display
fig, axes = plt.subplots(1, 2, figsize=(12, 5))
axes[0].imshow(cv2.cvtColor(shear_x, cv2.COLOR_BGR2RGB))
axes[0].set_title('Shear X (horizontal)')
axes[0].axis('off')

axes[1].imshow(cv2.cvtColor(shear_y, cv2.COLOR_BGR2RGB))
axes[1].set_title('Shear Y (vertical)')
axes[1].axis('off')

plt.tight_layout()
plt.show()

cv2.imwrite('images/shear_x.jpg', shear_x)
cv2.imwrite('images/shear_y.jpg', shear_y)
print("✅ Horizontal and vertical shear applied")

```



✅ Horizontal and vertical shear applied

Task2 — 1) Negative Image

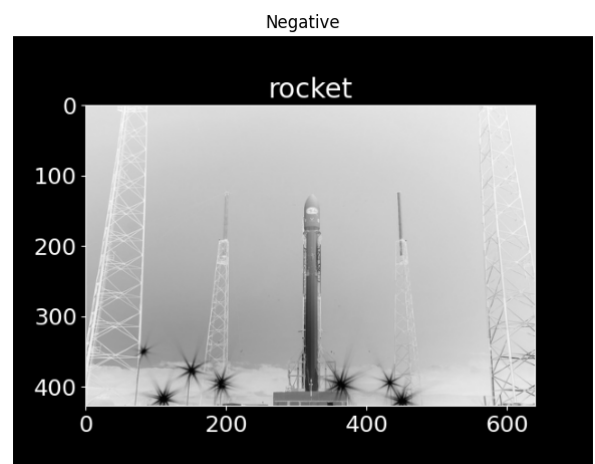
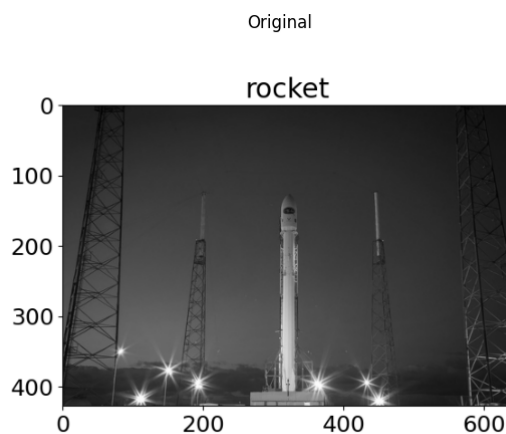
```
In [27]: # Task2 - 1) Create a Negative Image
import cv2

img = cv2.imread('images/input.jpg', cv2.IMREAD_GRAYSCALE)

negative = 255 - img

fig, axes = plt.subplots(1, 2, figsize=(12, 5))
axes[0].imshow(img, cmap='gray'); axes[0].set_title('Original');
axes[0].axis('off')
axes[1].imshow(negative, cmap='gray'); axes[1].set_title('Negative');
axes[1].axis('off')
plt.tight_layout()
plt.show()

cv2.imwrite('images/negative.jpg', negative)
print("✅ Negative image created")
```



✅ Negative image created

Task2 — 2) Log Transformation

```
In [28]: # Task2 - 2) Log Transformation
import cv2
import numpy as np

img = cv2.imread('images/input.jpg', cv2.IMREAD_GRAYSCALE)

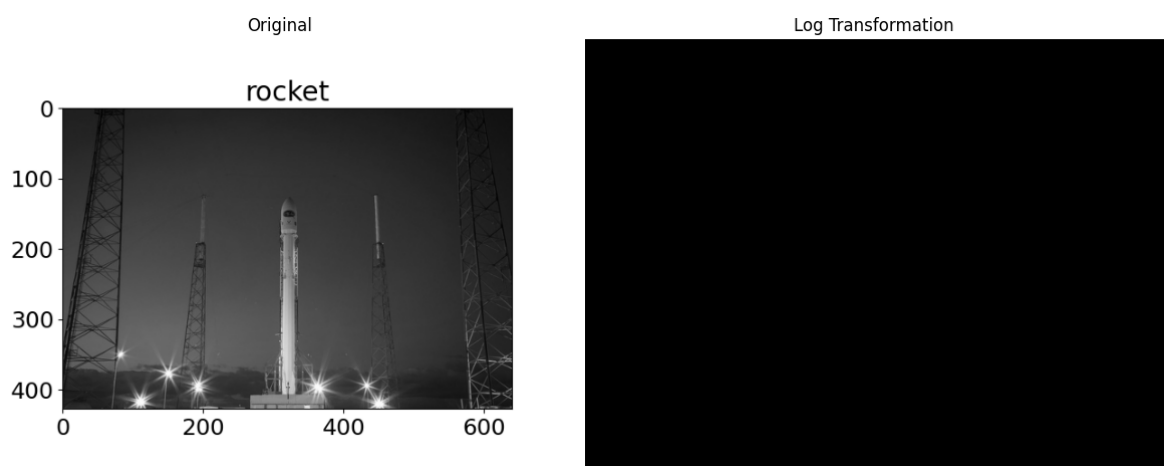
# c = 255 / log(1 + max_pixel)
c = 255 / np.log(1 + np.max(img))

log_img = c * np.log(1 + img.astype(np.float32))
log_img = np.array(log_img, dtype=np.uint8)

fig, axes = plt.subplots(1, 2, figsize=(12, 5))
axes[0].imshow(img, cmap='gray'); axes[0].set_title('Original');
axes[0].axis('off')
axes[1].imshow(log_img, cmap='gray'); axes[1].set_title('Log
Transformation'); axes[1].axis('off')
plt.tight_layout()
plt.show()

cv2.imwrite('images/log_transform.jpg', log_img)
print("✅ Log transformation applied, c =", c)
```

```
/var/folders/8j/0rv_vh_d2fv44vq1h9_1pjzc0000gn/T/ipykernel_5797/360937
3638.py:8: RuntimeWarning: overflow encountered in scalar add
  c = 255 / np.log(1 + np.max(img))
/var/folders/8j/0rv_vh_d2fv44vq1h9_1pjzc0000gn/T/ipykernel_5797/360937
3638.py:8: RuntimeWarning: divide by zero encountered in log
  c = 255 / np.log(1 + np.max(img))
```



✅ Log transformation applied, c = -0.0

Task2 - 3) Power Law (Gamma) - gamma that increases contrast

```
In [29]: # Task2 - 3) Power Law (Gamma) - gamma that increases contrast
import cv2
import numpy as np

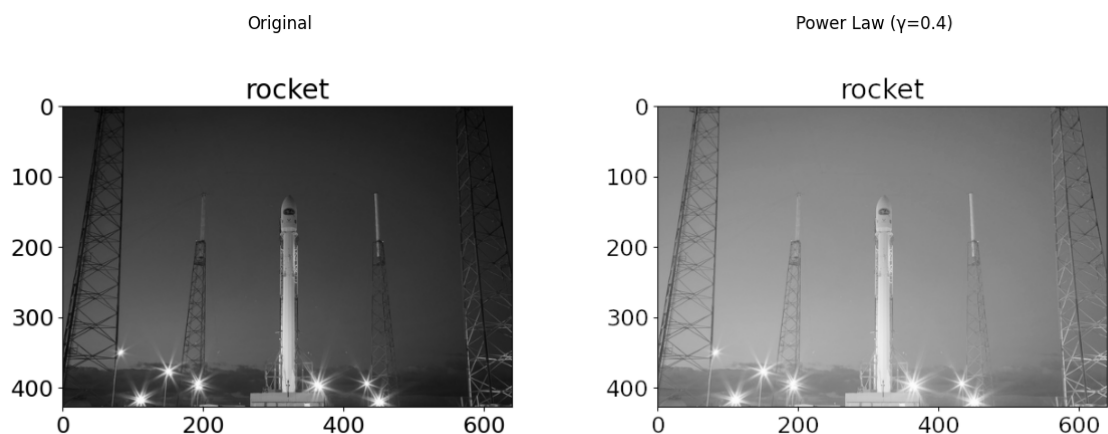
img = cv2.imread('images/input.jpg', cv2.IMREAD_GRAYSCALE)

gamma = 0.4 # value that increases contrast in dark regions

power_img = np.power(img / 255.0, gamma)
power_img = np.array(power_img * 255, dtype=np.uint8)

fig, axes = plt.subplots(1, 2, figsize=(12, 5))
axes[0].imshow(img, cmap='gray'); axes[0].set_title('Original');
axes[0].axis('off')
axes[1].imshow(power_img, cmap='gray'); axes[1].set_title(f'Power Law
( $\gamma={\gamma}$ )'); axes[1].axis('off')
plt.tight_layout()
plt.show()

cv2.imwrite('images/power_law.jpg', power_img)
print("✅ Power law applied with gamma =", gamma)
```



✅ Power law applied with gamma = 0.4